HEH.be Sciences et technologies

Bus de communication

Nano-ordinateurs

François Roland

WALLONIE-BRUXELLES ENSEIGNEMENT

pôle hainuyer

Notes

1 Introduction

2 Bus parallèle

3 UART

4 i2c

5 Conclusion

Notes

Complexité des données

Données simples `0` ou `1`, allumé ou éteint, présent ou absent...

Données complexes distance, température, accélération...

Notes

Capteur de distance

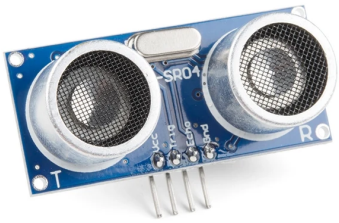


Figure : Capteur de distance HC-SR04

Solution du fabricant largeur d'impulsion → distance

Alternative 14 cm → `00001110`

Notes

Câblage transmission binaire

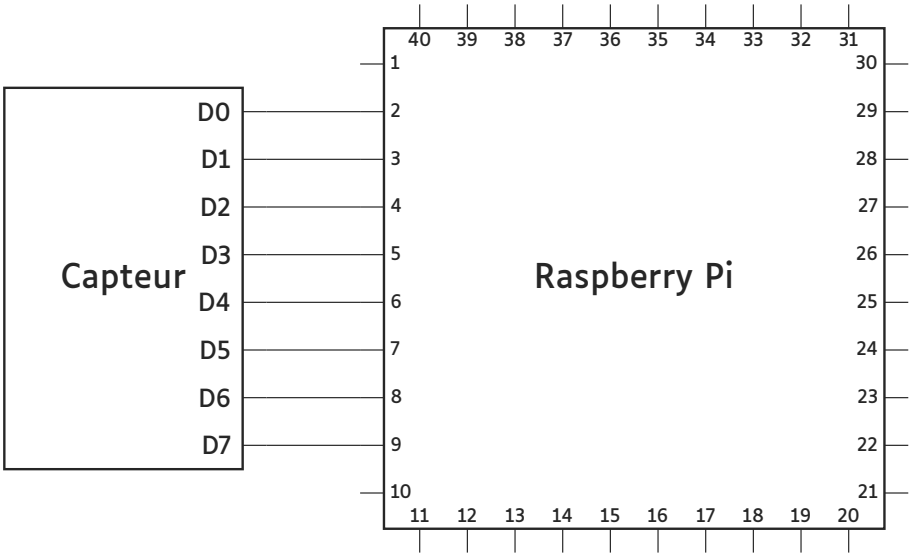


Figure : Câblage 8 bits entre un capteur et un Raspberry Pi

Notes

Plusieurs capteurs

- 1 capteur ⇒ 8 lignes de communication
- 2 capteurs ⇒ 16 lignes de communication
- 3 capteurs ⇒ 24 lignes de communication
- ...

Notes

Bus de données

Définition

Un bus est un ensemble de lignes de communication permettant l'échange de données entre plusieurs composants d'un système informatique.

Notes

Comparaison des bus de communication

Axe	Question centrale
Débit	Combien de bits simultanément ?
Synchronisation	Comment sait-on quand lire un bit ?
Adressage	Comment identifier le destinataire ?
Direction	Qui peut parler ? Quand ?
Câblage	Combien de fils ?

Notes

1 Introduction

2 Bus parallèle

3 UART

4 i2c

5 Conclusion

Notes

Bus parallèle

Définition

Un bus parallèle est un bus qui transmet plusieurs bits simultanément sur plusieurs lignes de communication.

Exemple : bus entre le CPU et la RAM d'un ordinateur

Notes

Schéma d'un bus parallèle

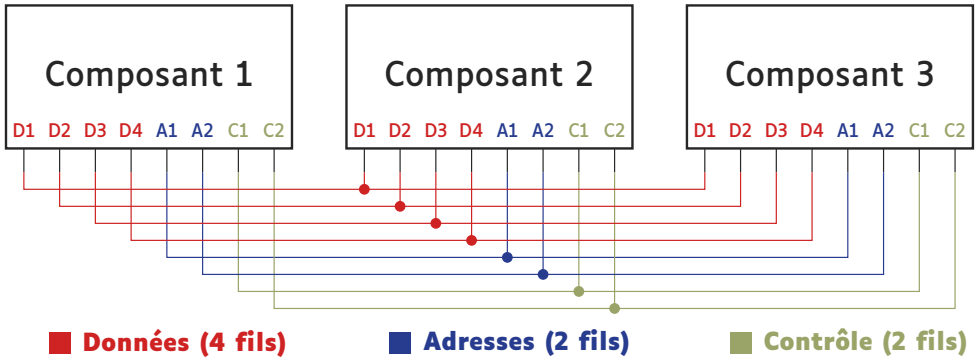


Figure : Exemple de bus parallèle avec 4 bits de données, 2 bits d'adresses et 2 bits de contrôle

Notes

Limites physiques

- Nombre de lignes de communication
- Interférences électromagnétiques à haute fréquence
- Longueur identique des lignes de communication

Notes

Pistes de longeur identique

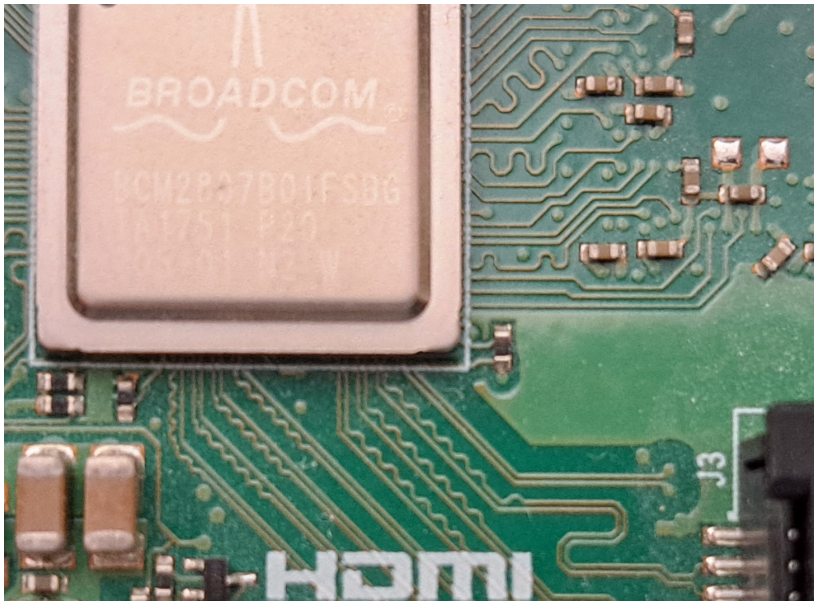


Figure : Lignes de délai sur un Raspberry Pi 3B+

Notes

Caractéristiques du bus parallèle

Axe	**Bus parallèle**
Débit	N bits simultanément (N = largeur du bus)
Synchronisation	?
Adressage	Bus d'adresse dédié
Direction	Bus de contrôle dédié (lecture/écriture)
Câblage	N fils de données + adresses + contrôle

Notes

1 Introduction

2 Bus parallèle

3 UART

4 i2c

5 Conclusion

Notes

Bus série

Définition

Un bus série est un bus qui transmet les données bit par bit sur une seule ligne de communication.

Exemple

USB, SATA, PCIe, i^2c, SPI, UART

Notes

Bus UART

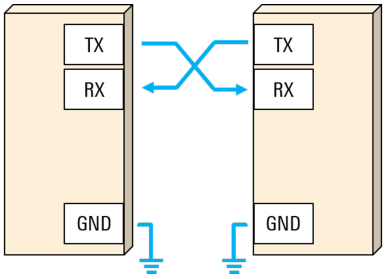


Figure : Rohde & Schwarz, https://www.rohde-schwarz.com/nl/products/test-and-measurement/essentials-test-equipment/digital-oscilloscopes/understanding-uart_254524.html, consulté le 2025-02-09

- Bus série asynchrone bidirectionnel
- 2 fils : RX et TX
- Point-à-point (max 2 participants)
- Simplex, half-duplex ou full-duplex
- Vitesses de transfert entre 480 bps et 115 200 bps

Notes

Configuration

Vitesse de transmission	baud rate = vitesse de transmission des signaux
Nombre de bits de données	de 5 à 9
Bit de parité	aucun (N), pair (E) ou impair (O) = détection d'erreur
Nombre de bits de STOP	1 ou 2

Exemple

- 9600-8-N-1 : 9600 baud, 8 bits de données, pas de bit de parité, 1 bit de STOP.
- 115200-7-E-2 : 115200 baud, 7 bits de données, bit de parité pair, 2 bits de STOP.

Notes

Codage des données

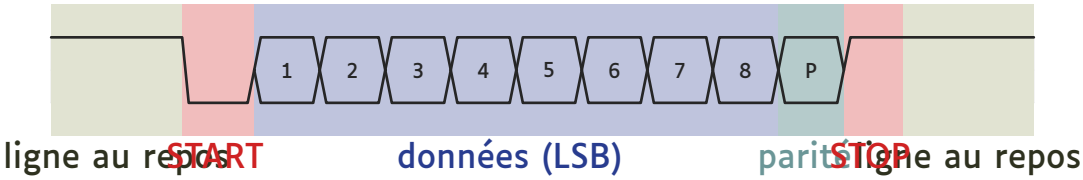


Figure : Codage UART 8 bits + 1 bit de parité + 1 bit de stop (???-8-E-1)

Notes

Signaux

Transformation des bits (`0` et `1`) en signaux électriques.

Standards de signalement les plus courants :

- **TTL** standard pour les communications série sur des courtes distances
- **RS-232** standard historique pour les communications série
- **RS-485** standard pour les communications série en réseau

Notes

Signal TTL

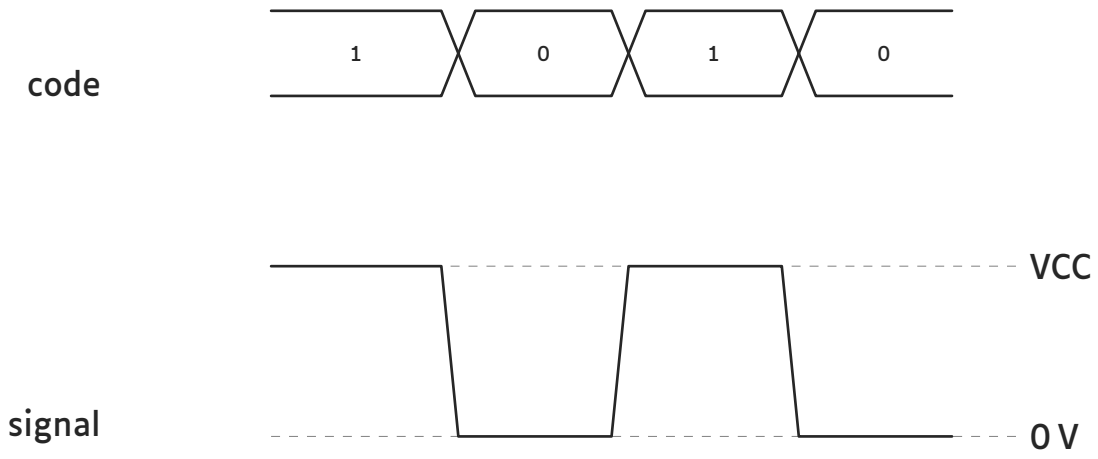


Figure : Encodage d'un signal TTL

Notes

Signal RS-232

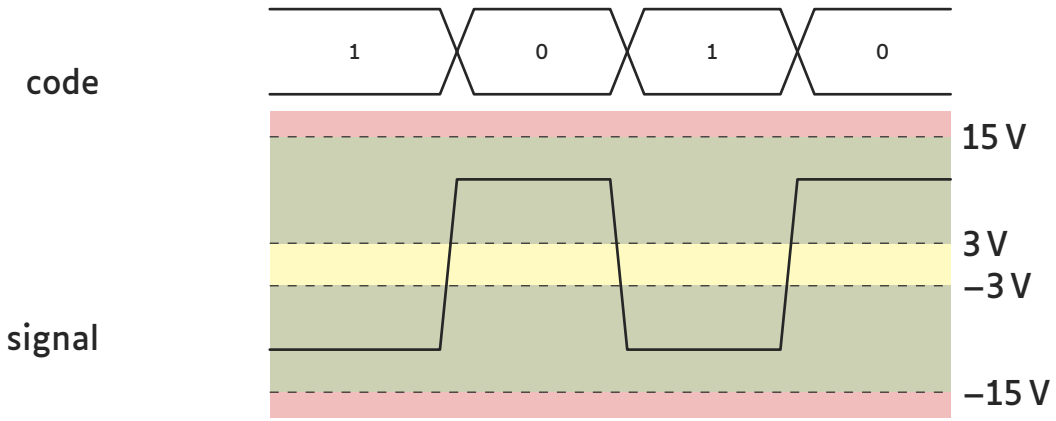


Figure : Encodage d'un signal RS-232

Notes

Signal RS-485

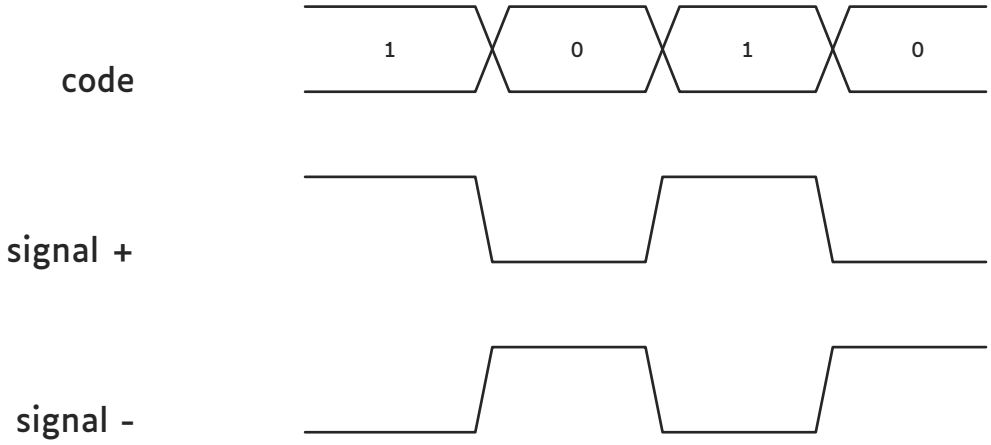


Figure : Encodage d'un signal RS-485

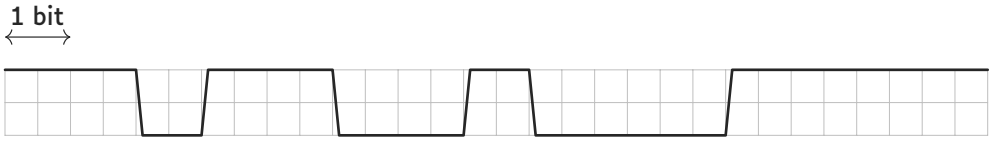
- 2 paires torsadées pour les données (TX+, TX-, RX+, RX-)
- Signal valide ≥ 1,5 V entre 2 fils d'une même paire
- 1 fil de masse ⇒ ⚠ boucles de terre sur longues distances

Notes

Décodage d'une communication UART

Exercice

signal



Voici un signal UART TTL 115200-7-E-2 capturé par un analyseur logique.
Quel nombre a été transmis ?

Notes

Démonstration

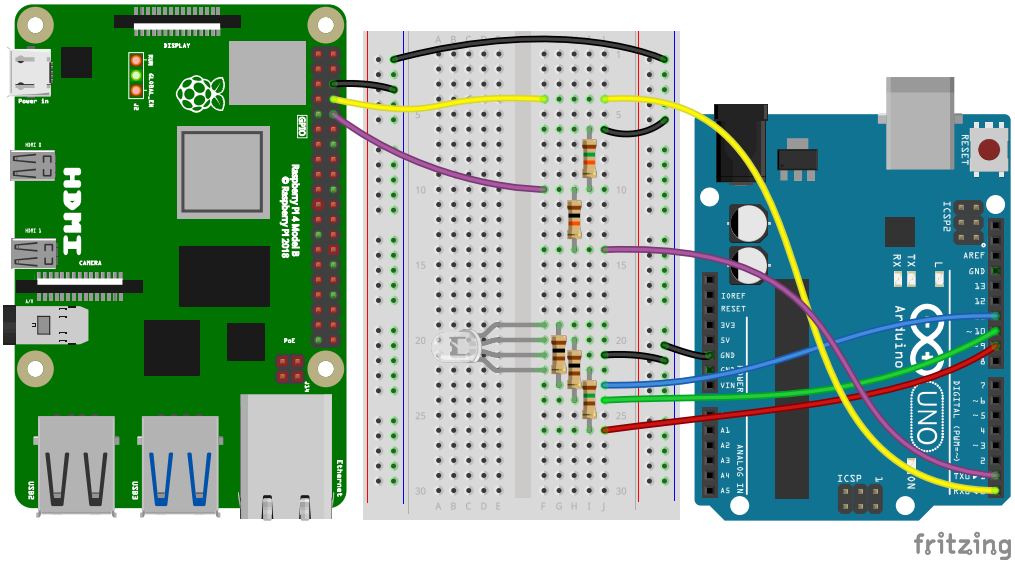


Figure : Communication UART entre un Raspberry Pi et un Arduino

Notes

Code Arduino

```
void setup() {
  Serial.begin(9600, SERIAL_8N1);
  pinMode(9, OUTPUT);   // Rouge
  pinMode(10, OUTPUT);  // Vert
  pinMode(11, OUTPUT);  // Bleu
}

void loop() {
  if (Serial.available()) {
    int r = Serial.parseInt();
    Serial.read();  // Lit l'espace
    int g = Serial.parseInt();
    Serial.read();  // Lit l'espace
    int b = Serial.parseInt();
    Serial.read();  // Lit le \n
    analogWrite(9, r);
    analogWrite(10, g);
    analogWrite(11, b);
    Serial.print("OK ");
    Serial.print(r); Serial.print(" ");
    Serial.print(g); Serial.print(" ");
    Serial.println(b);
  }
}
```


Notes

Code Raspberry Pi

```python
import serial
from time import sleep

COULEURS = [
    (255, 0, 0),  # rouge
    (0, 128, 0),  # vert
    (0, 0, 255),  # bleu
    (255, 20, 147),  # rose
    (255, 69, 0),  # orange
    (0, 0, 0),  # éteint
]

with serial.Serial("/dev/serial0", baudrate=9600, timeout=1) as uart:
    for r, g, b in COULEURS:
        commande = f"{r} {g} {b}\n"
        uart.write(commande.encode())
        reponse = uart.readline().decode().strip()
        print(f"Envoyé : {commande.strip()} - Reçu : {reponse}")
        sleep(1)
```


Notes

Caractéristiques du bus UART

Axe	Bus UART
Débit	1 bit à la fois, jusqu'à 115 200 bps
Synchronisation	Baud rate identique des deux côtés
Adressage	Aucun (point-à-point)
Direction	Full-duplex (RX et TX séparés)
Câblage	2 fils : TX et RX

Notes

Et si on veut plusieurs destinataires ?

UART limité à 2 participants

Pour connecter plusieurs composants sur le même bus, il faut :

horloge commune pour synchroniser les participants

adresses pour identifier le destinataire

→ bus i^2c

Notes

1 Introduction

2 Bus parallèle

3 UART

4 i2c

5 Conclusion

Notes

Bus i^2c

- Bus série synchrone bidirectionnel
- 2 fils : données (SDA) et horloge (SCL)
- Contrôlé par un composant « maître »
- Plusieurs « esclaves » possibles
- Vitesse de transfert entre 100 kbps et 5 Mbps

Notes

Début, fin et bits de données

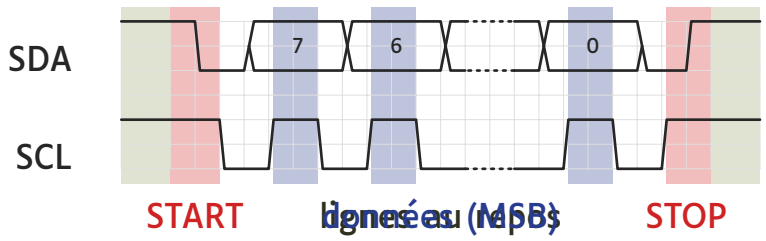


Figure : Codage i^2c

Notes

Envoi d'un octet par le maître

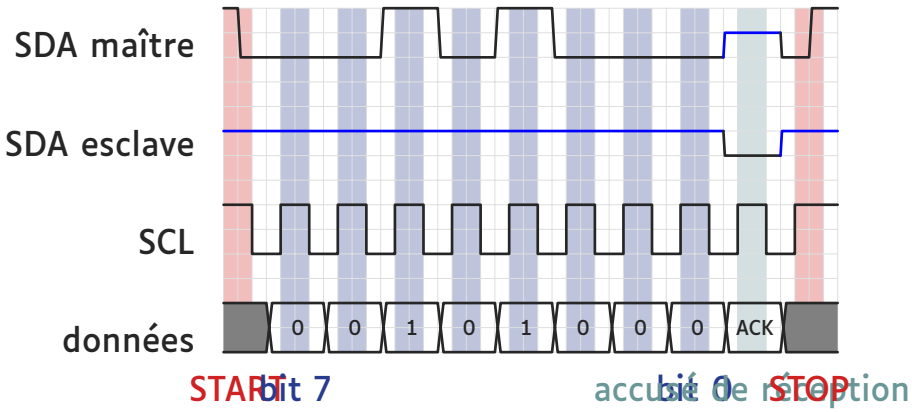


Figure : Envoi par le maître avec accusé de réception par l'esclave

Notes

Écriture d'un octet

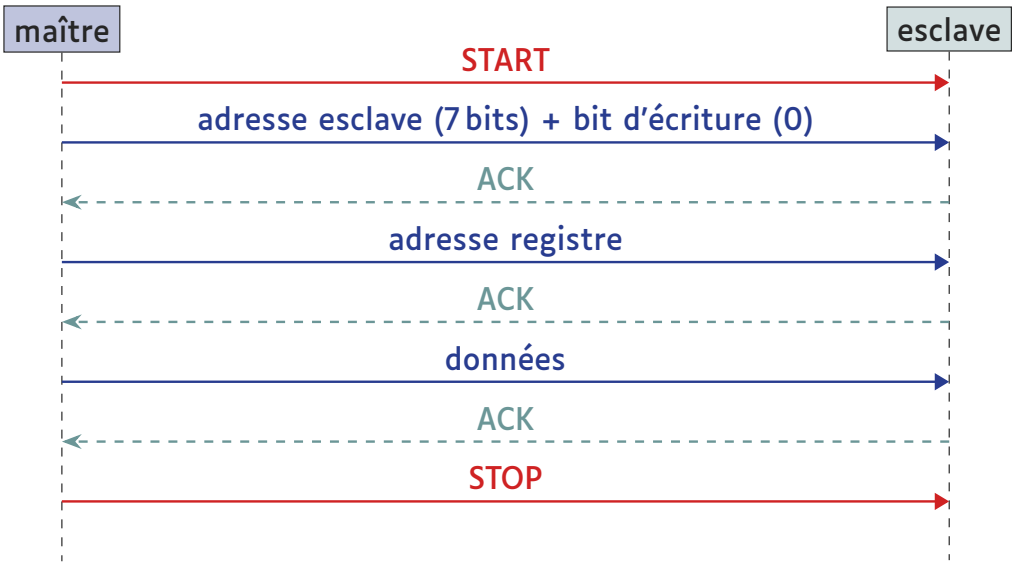


Figure : Écriture d'un octet par le maître vers un registre de l'esclave

Notes

Lecture d'un octet

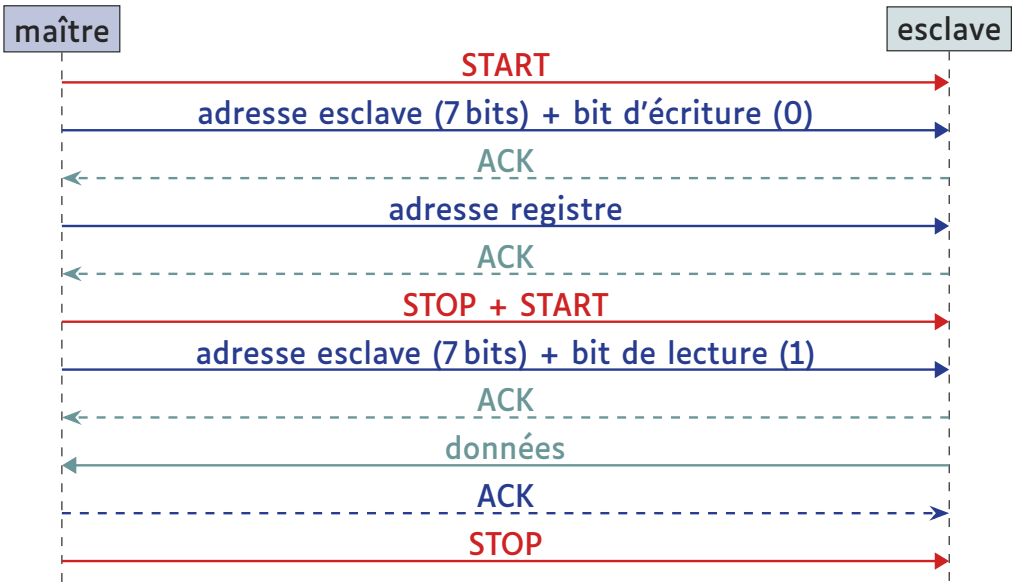


Figure : Lecture d'un octet par le maître depuis un registre de l'esclave

Notes

Résumé de la structure des communications

- Écriture

0 1 2 3 4 5 6	7	8 9 10 11 12 13 14 15	16 17 18 19 20 21 22 23
adresse esclave	W (0)	adresse registre	données à écrire dans le registre

- Lecture

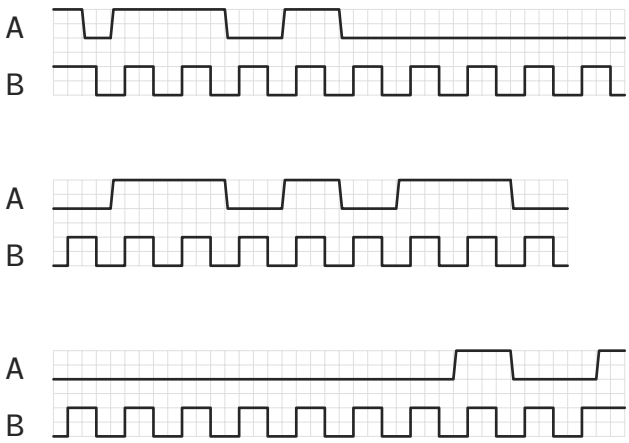
0 1 2 3 4 5 6	7	8 9 10 11 12 13 14 15
adresse esclave	W (0)	adresse registre
adresse esclave	R (1)	données lues depuis le registre

Notes

Décodage d'une communication i^2c

Exercice

Capture des signaux A et B par un analyseur logique :

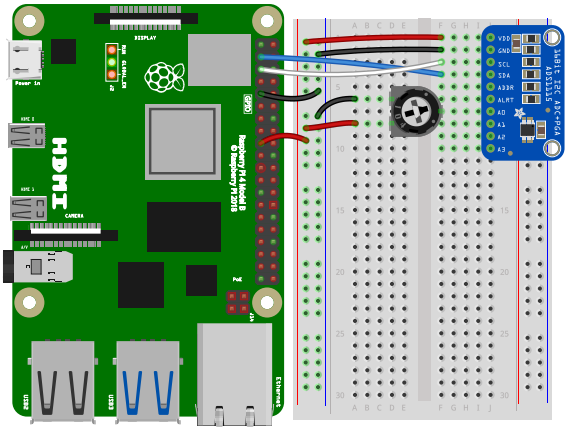


- SDA ou SCL ?
- Adresse de l'esclave ?
- Lecture ou écriture ?
- Adresse du registre ?
- Valeur échangée ?

Notes

Démonstration i^2c

Convertisseur analogique-numérique



1. Connecter le RPi et le convertisseur analogique-numérique sur le bus i^2c
2. Exécuter le programme
3. Observer les signaux avec l'analyseur logique

Notes

Code Raspberry Pi

```python
from smbus2 import SMBus
from time import sleep

with SMBus(1) as bus:
    # Configure l'ADC (adresse 0x48)
    bus.write_i2c_block_data(0x48, 0x01, [0xC2, 0x83])
    sleep(0.1)

    while True:
        # Lit 2 octets depuis le registre 0x00 de l'ADC
        bytes = bus.read_i2c_block_data(0x48, 0x00, 2)
        # Combine la partie haute et la partie basse
        value = (bytes[0] << 8) | bytes[1]
        print(value)
        sleep(0.5)
```


Notes

Lister les esclaves du bus i^2c

Astuce

```
sudo i2cdetect -y 1
```

→ adresses des esclaves connectés au bus i^2c

Notes

Caractéristiques du bus i^2c

Axe	**Bus** i^2c
Débit	100 kbps à 5 Mbps
Synchronisation	Signal d'horloge (SCL) généré par le maître
Adressage	7 bits
Direction	Half-duplex (SDA commun)
Câblage	2 fils : SDA et SCL

Notes

1 Introduction

2 Bus parallèle

3 UART

4 i2c

5 Conclusion

Notes

Comparaison des bus de communication

Axe	**Parallèle**	**UART**	i^2c
Débit	N bits simultanément	≤115 kbps	≤5 Mbps
Synchronisation	Horloge commune	Baud rate identique	Signal SCL (maître)
Adressage	Bus d'adresse dédié	Aucun	7 bits
Direction	Bus de contrôle dédié	Full-duplex	Half-duplex
Câblage	100+ fils	2 fils	2 fils

Notes

Quel bus choisir ?

UART communication point-à-point simple

i^2c plusieurs capteurs sur le même bus

Souvent le choix est imposé par les composants que l'on utilise.

Et SPI ?

- Plus rapide que i^2c
- Sans adressage natif, nécessite un fil de sélection pour chaque esclave
- Full-duplex

Notes

Résumé

- Différents bus de communication et leurs caractéristiques
- Encodage données ↔ signal

Prochaine séance : traitement de données en entrées et en sortie

Notes